

ENG3090 Lec 3b: images processing

Massimo Vassalli

massimo.vassalli@glasgow.ac.uk

Images processing

Image processing is at the heart of modern data analysis, computer vision, and biomedical imaging. At its core, image processing is about transforming raw pixel values into information: reducing noise, enhancing contrast, detecting edges, or highlighting structures of interest. One of the most powerful and universal tools in this process is the filter.

Filters allow us to modify images in a controlled way, from smoothing out random variations to emphasizing fine details. Whether we are blurring, sharpening, or detecting edges, most of these operations can be described through a single unifying mathematical principle: convolution with a kernel. By understanding filters as kernels applied across an image, we gain a common language for many seemingly different image processing techniques.

What is a Filter?

- In image processing, a **filter** is an operation that modifies an image to enhance or extract specific features.
- Filters can:
 - Reduce noise
 - Enhance edges
 - Blur or sharpen details
 - Detect patterns or gradients
- Mathematically, many filters are based on **convolution**.

The image as a mathematical representation

Images (grayscale) can be considered as a continuous manifold $\mathcal{I}$ defined over the spatial coordinates (x, y) and sampled over a discrete array of values (x_i, y_i) :

$$\mathcal{I}_{i,j} = \mathcal{I}(x_i, y_i)$$

127	112	...	...	...					
					75				
				95	$\mathcal{I}_{i,j}$	22			
					184				

Filter categories

Filters transform images through a function T

$$\mathcal{I} \xrightarrow{T} \mathcal{G}$$

The value of each destination pixel $\mathcal{G}_{i,j}$ upon filtering typically depends either on the entire image values (global filters) or a neighborhood of the source pixel (local filters).

Convolution in Image Processing

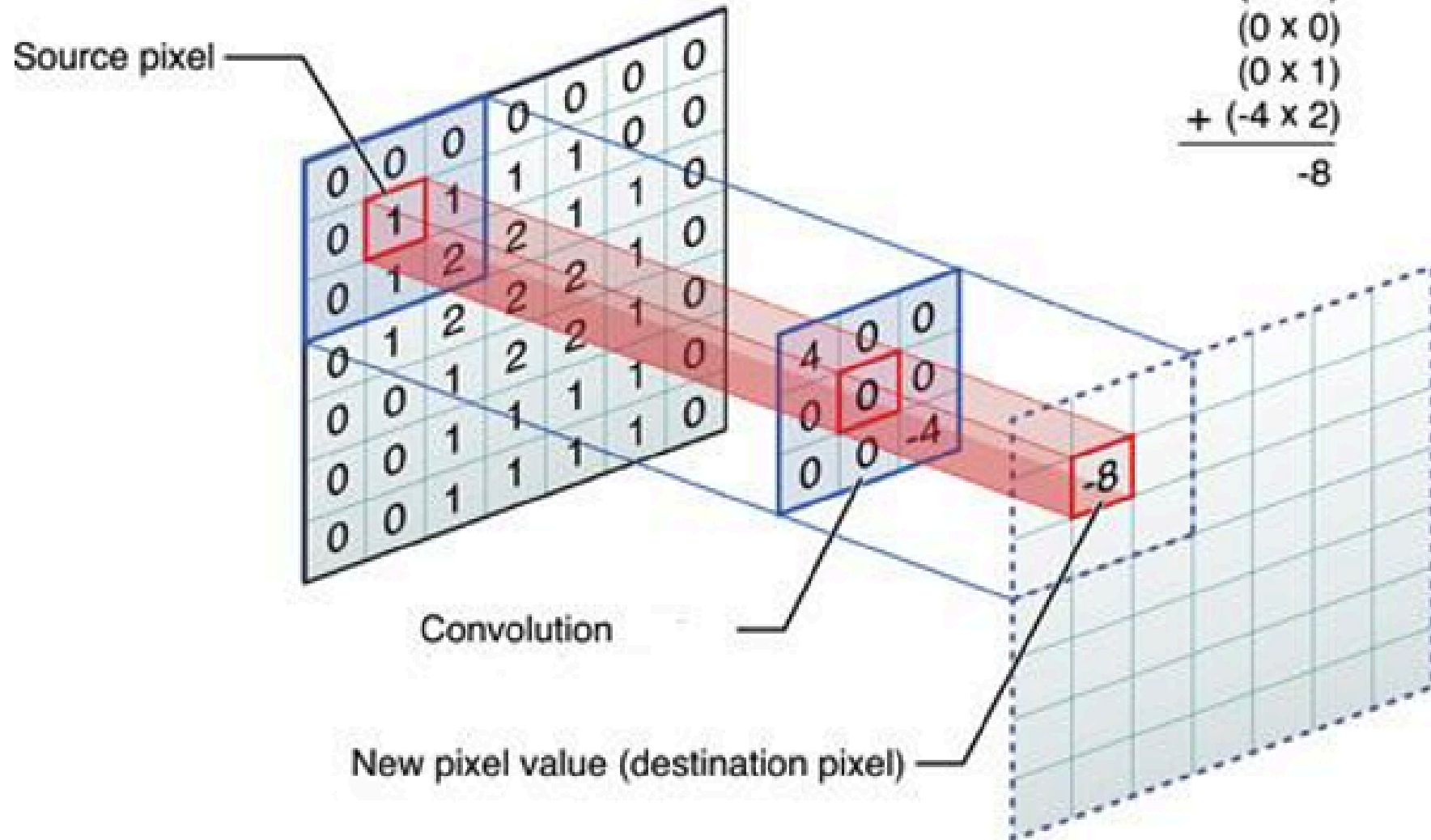
- **Convolution** = applying a small matrix (kernel) to each pixel and its neighborhood.
- The kernel defines how the pixel values are combined.

$$\mathcal{G}(x, y) = \sum_{i=-m}^m \sum_{j=-n}^n \mathcal{I}(x + i, y + j) \cdot k(i, j)$$

- $\mathcal{I}(x, y)$: input image
- $k(i, j)$: filter kernel
- $\mathcal{G}(x, y)$: filtered image

Center element of the kernel is placed over the source pixel. The source pixel is then replaced with a weighted sum of itself and nearby pixels.

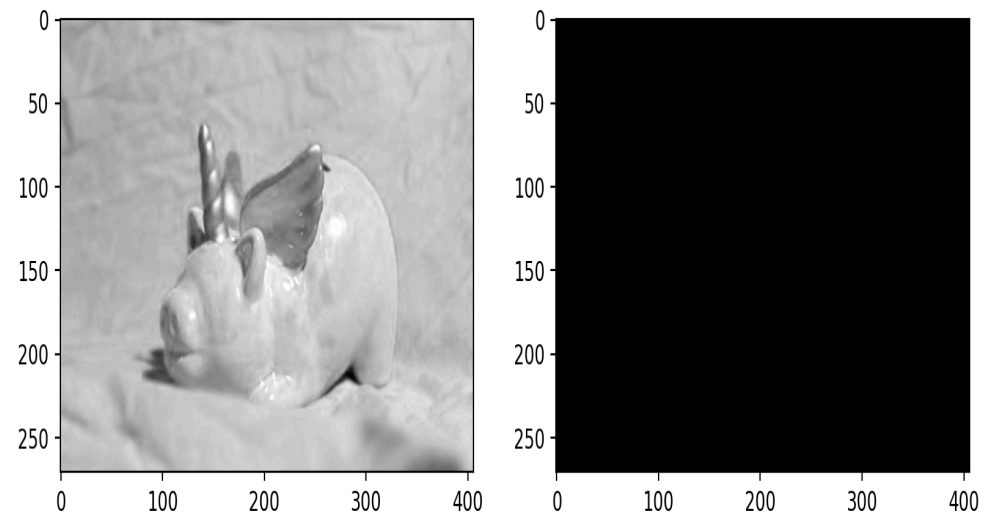
$$\begin{array}{r}
 (4 \times 0) \\
 (0 \times 0) \\
 (0 \times 0) \\
 (0 \times 0) \\
 (0 \times 1) \\
 (0 \times 1) \\
 (0 \times 0) \\
 (0 \times 1) \\
 \hline
 + (-4 \times 2) \\
 \hline
 -8
 \end{array}$$



simple convolutions

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from skimage import io
4 from scipy.ndimage import convolve
5 image = io.imread('img/unipig.jpg')
6 red = image[:, :, 0].astype(float)
7
8 kernel = np.zeros((3, 3))
9 cred = convolve(red, kernel)
10
11 fig, axs = plt.subplots(1, 2, figsize=(10, 4))
12 axs[0].imshow(red, cmap='gray')
13 axs[1].imshow(cred, cmap='gray')
```

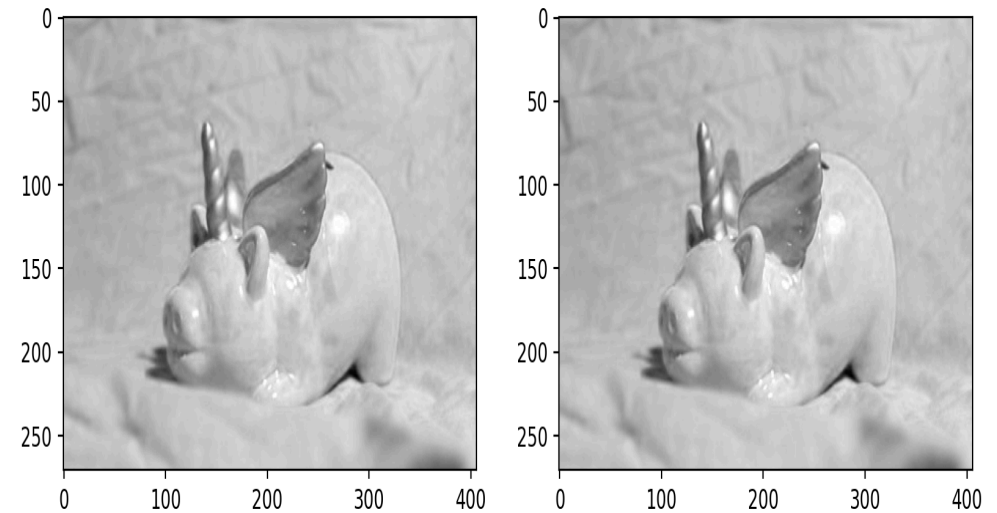


$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

```

1 kernel = np.zeros((3,3))
2 kernel[1,1]=1
3 cred = convolve(red,kernel)
4
5 fig,axs=plt.subplots(1,2,figsize=(10,4))
6 axs[0].imshow(red,cmap='gray')
7 axs[1].imshow(cred,cmap='gray')

```

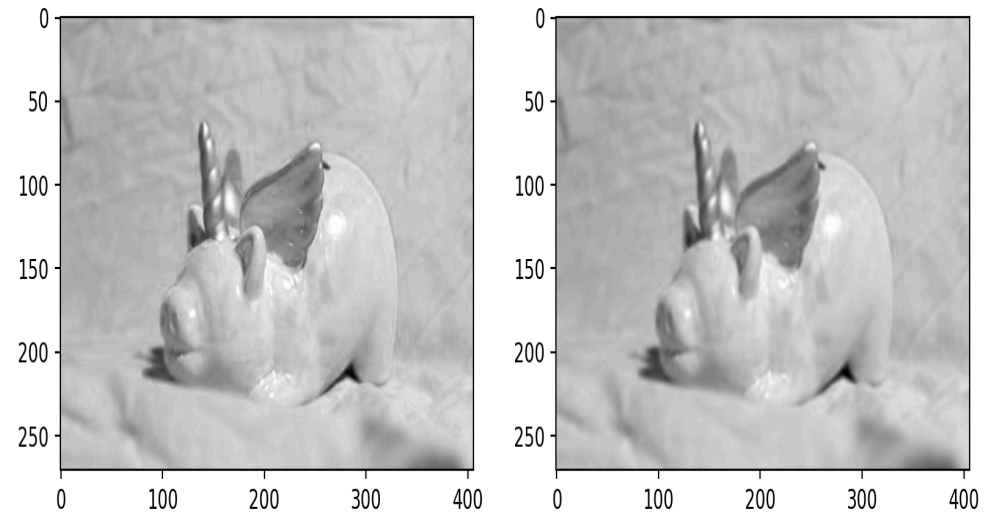


$$\mathcal{G}_{i,j} = \mathcal{I}_{i,j}$$

The unity matrix corresponds to identity in convolution

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

```
1 kernel = np.ones((3,3))
2 cred = convolve(red,kernel)
3
4 fig,axs=plt.subplots(1,2,figsize=(10,4))
5 axs[0].imshow(red,cmap='gray')
6 axs[1].imshow(cred,cmap='gray')
```



$$\mathcal{G}(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 \mathcal{I}(x + i, y + j) \cdot 1$$

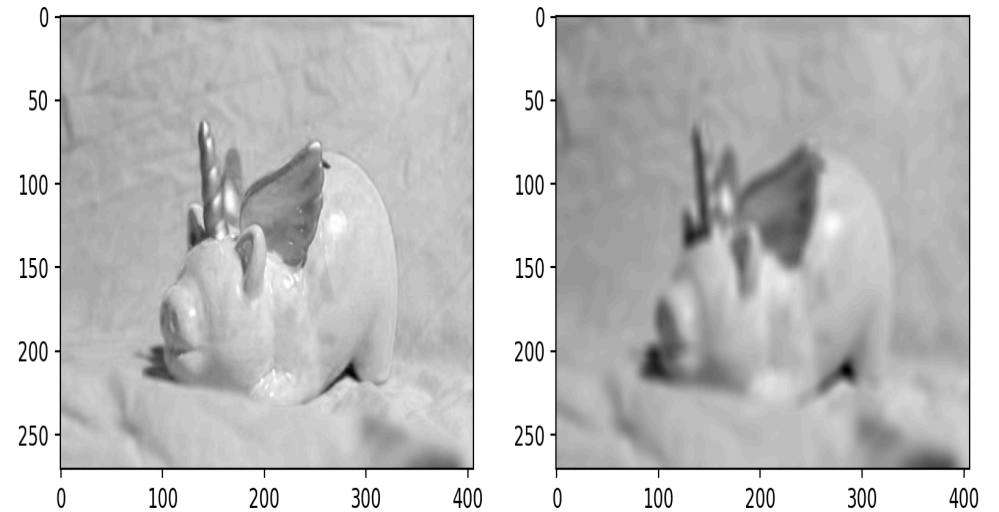
i Note

Transformation kernels are normalised to 1 (one)

```

1 kernel = np.ones((9,9))
2 cred = convolve(red,kernel)
3
4 fig,axs=plt.subplots(1,2,figsize=(10,4))
5 axs[0].imshow(red,cmap='gray')
6 axs[1].imshow(cred,cmap='gray')

```



$$\mathcal{G}(x, y) = \langle \mathcal{I}(x', y') \rangle_K$$

The unity matrix performs the average (smoothing) of the image

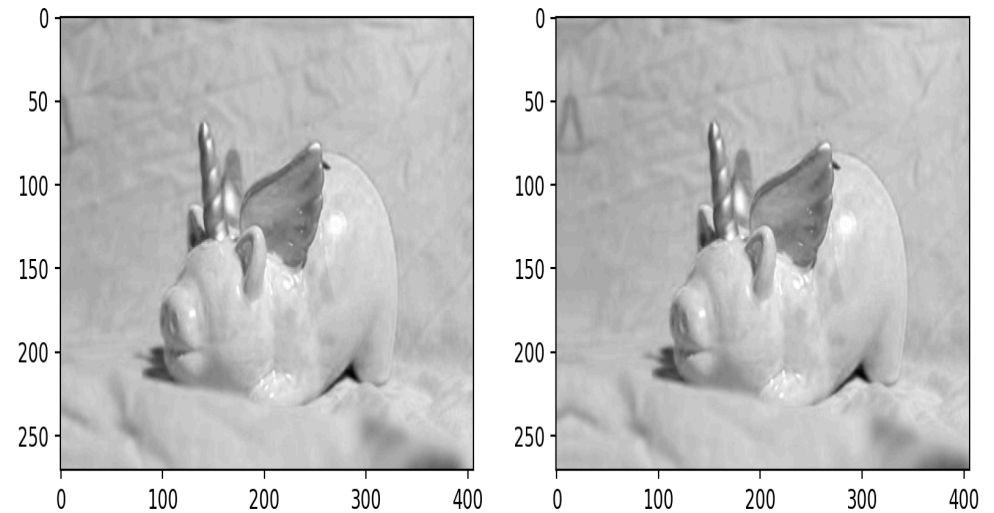
$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\mathcal{G}(x, y) = \mathcal{I}(x + 1, y)$$

```

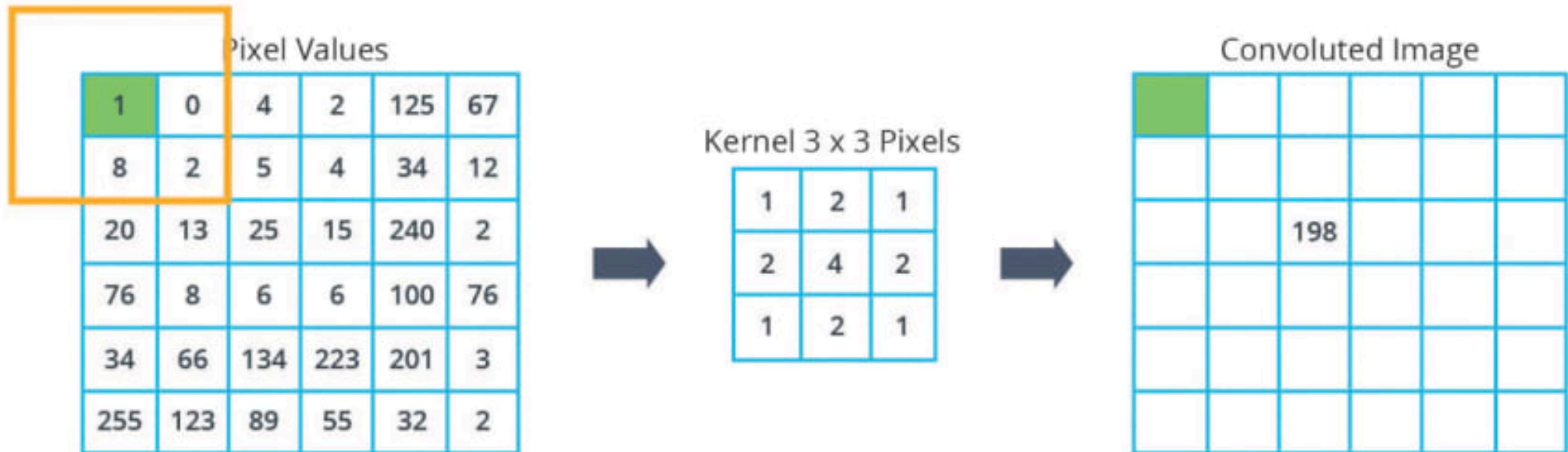
1  L=14
2  kernel = np.zeros((2*L+1,2*L+1))
3  kernel[L,-1]=1
4  cred = convolve(red,kernel)
5
6  fig,axs=plt.subplots(1,2,figsize=(10,4))
7  axs[0].imshow(red,cmap='gray')
8  axs[1].imshow(cred,cmap='gray')

```

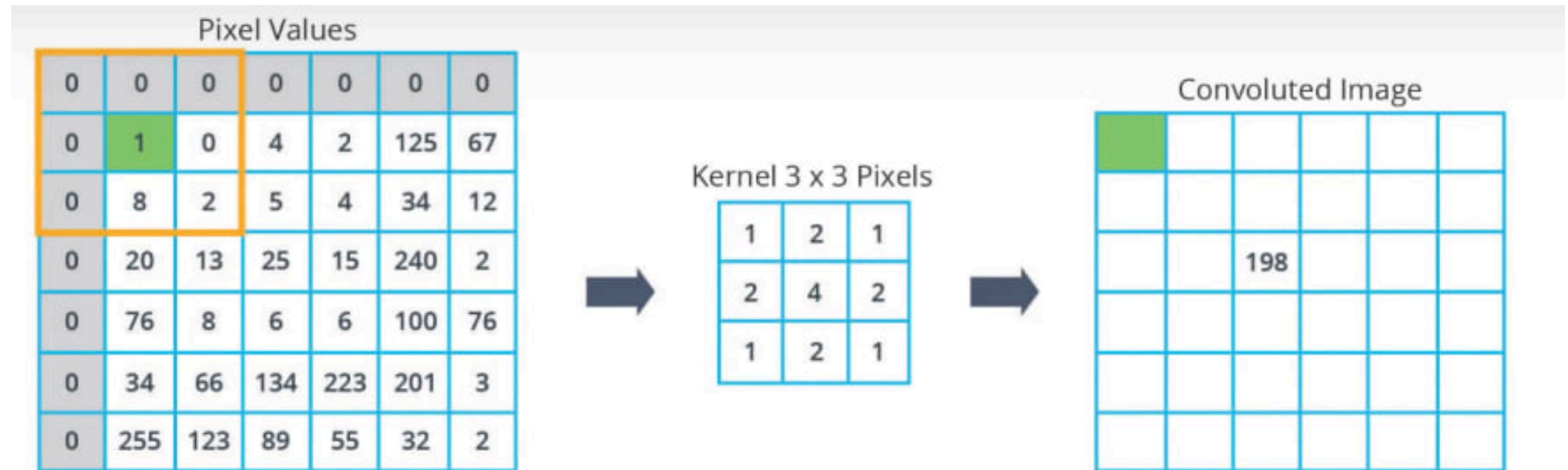


shift horizontally

Missing pixels



The default behaviour is to extend the matrix, filling the gray spots with values based on a rule



mode{‘reflect’, ‘constant’, ‘nearest’, ‘mirror’, ‘wrap’}, optional

The mode parameter determines how the input array is extended beyond its boundaries. Default is ‘reflect’. Behavior for each valid value is as follows:

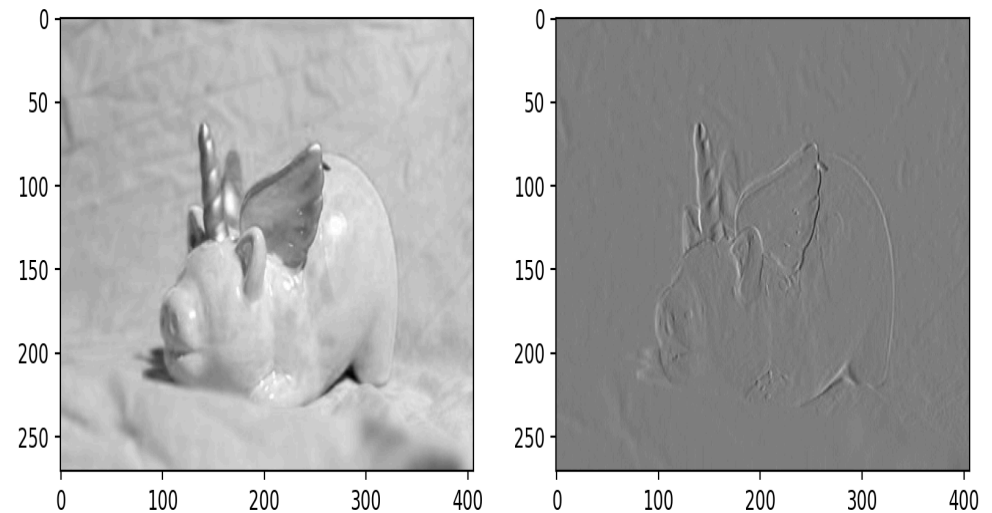
- ‘reflect’ (d c b a | a b c d | d c b a) The input is extended by reflecting about the edge of the last pixel. This mode is also sometimes referred to as half-sample symmetric.
- ‘constant’ (k k k k | a b c d | k k k k) The input is extended by filling all values beyond the edge with the same constant value, defined by the cval parameter.
- ‘nearest’ (a a a a | a b c d | d d d d) The input is extended by replicating the last pixel.
- ‘mirror’ (d c b | a b c d | c b a) The input is extended by reflecting about the center of the last pixel. This mode is also sometimes referred to as whole-sample symmetric.
- ‘wrap’ (a b c d | a b c d | a b c d) The input is extended by wrapping around to the opposite

Zero-sum kernels

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad \mathcal{G}(x, y) = \sum_{i=-m}^m \sum_{j=-n}^n \mathcal{I}(x + i, y + j) \cdot k(i, j)$$

$$\mathcal{G}(x, y) = \mathcal{I}(x + 1, y) - \mathcal{I}(x, y)$$

```
1 kernel = np.zeros((3,3))
2 kernel[1,2]=1
3 kernel[1,1]=-1
4 cred = convolve(red,kernel)
5 fig,axs=plt.subplots(1,2,figsize=(10,4))
6 axs[0].imshow(red,cmap='gray')
7 axs[1].imshow(cred,cmap='gray')
```

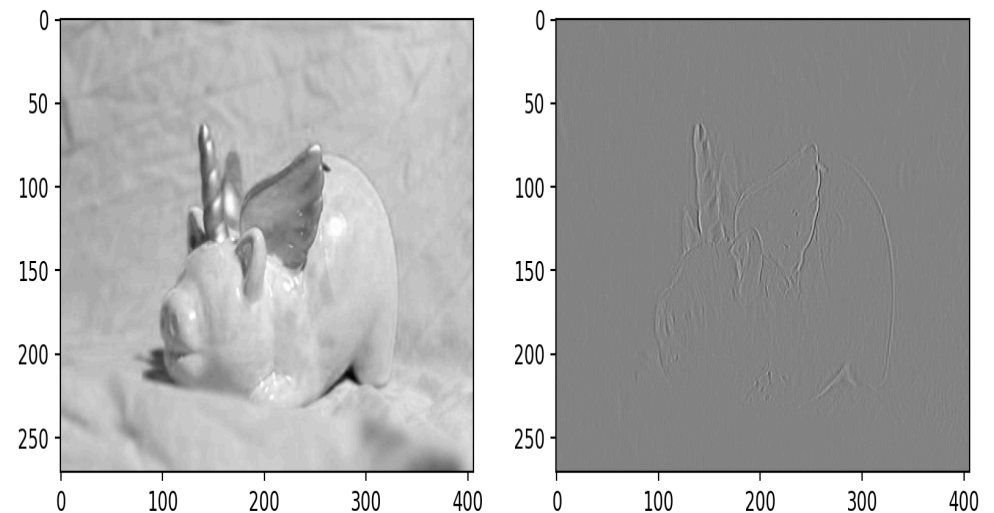


Differential kernel $\mathcal{G}(x, y) = \frac{\partial}{\partial x} \mathcal{I}(x, y)$

Nested differentiation

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

```
1 kernel = np.zeros((3,3))
2 kernel[1,2]=1
3 kernel[1,1]=-2
4 kernel[1,0]=1
5 cred = convolve(red,kernel)
6 fig,axs=plt.subplots(1,2,figsize=(10,4))
7 axs[0].imshow(red,cmap='gray')
8 axs[1].imshow(cred,cmap='gray')
```



Differential kernel $\mathcal{G}(x, y) = \frac{\partial^2}{\partial^2 x} \mathcal{I}(x, y)$

skimage.filters

Chaining convolutions together

The scikit-image package does not provide a general implementation of the `convolve` function, but it offers already packaged filters with optimised kernels.

In practice, `scikit-image` exposes high-level filters; for custom kernels you can use `scipy.ndimage.convolve` (with flip) or `scipy.ndimage.correlate` (no flip).

Note

Convolution and **Correlation** are similar operations. Convolution flips the kernel horizontally and vertically before sliding, while correlation uses the kernel **as-is**. Many libraries implement filters via correlation for convenience. For **symmetric kernels** (e.g., Gaussian), convolution == correlation. For **asymmetric kernels** (e.g., Sobel X), the flip matters.

Mathematically:

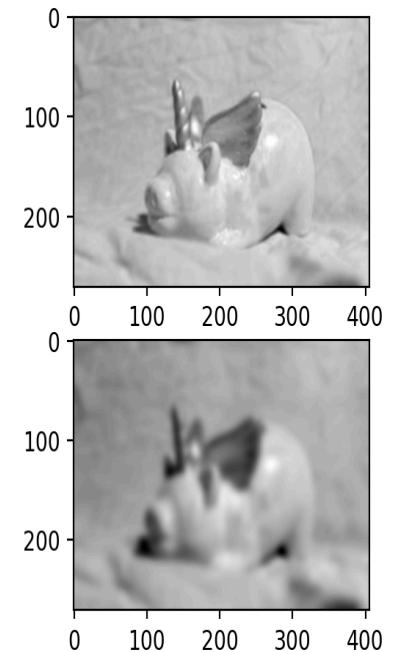
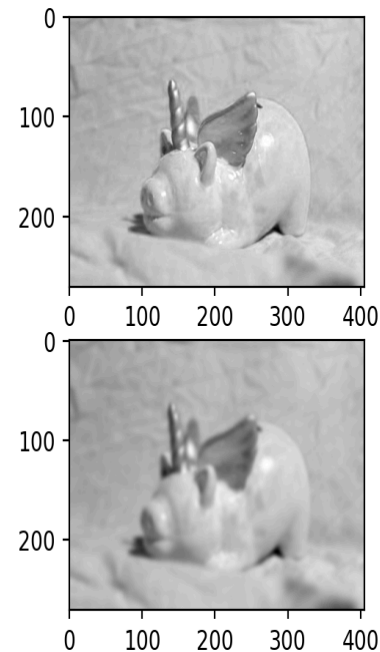
$$(g * h)(x, y) = \sum_{i,j} f(x - i, y - j)h(i, j),$$

$$(g \star h)(x, y) = \sum_{i,j} f(x + i, y + j)h(i, j)$$

Gaussian Filter (Convolution, scikit-image)

- Smooths images by weighting nearby pixels more strongly.
- Kernel looks like a 2D Gaussian distribution.

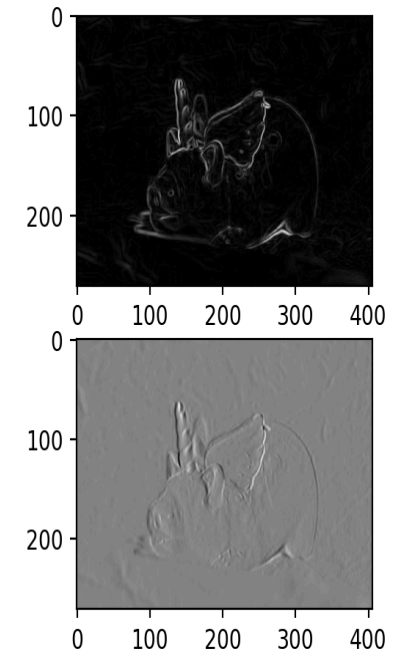
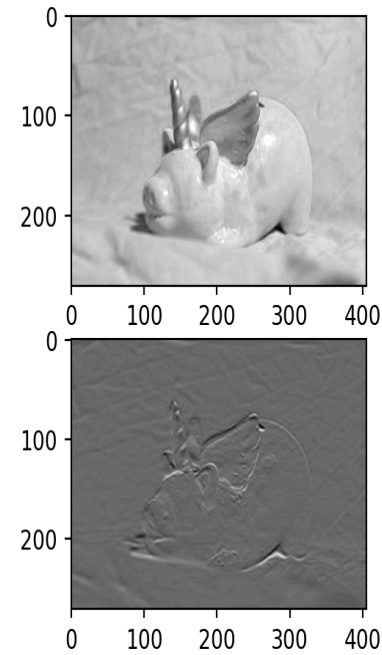
```
1 from skimage.filters import gaussian
2
3 blur=[]
4 for sig in [1.0,2.0,4.0]:
5     blur.append(gaussian(red, sigma=sig, preserve_mode=True))
6
7 fig,axs=plt.subplots(2,2,figsize=(10,4))
8 axs[0,0].imshow(red,cmap='gray')
9 j=0
10 for ax in axs.flatten()[1:]:
11     ax.imshow(blur[j],cmap='gray')
12     j+=1
```



Edge Detection as Convolution (scikit-image)

- Edge filters (such as Sobel, Prewitt) are **differential kernels** approximating image derivatives.

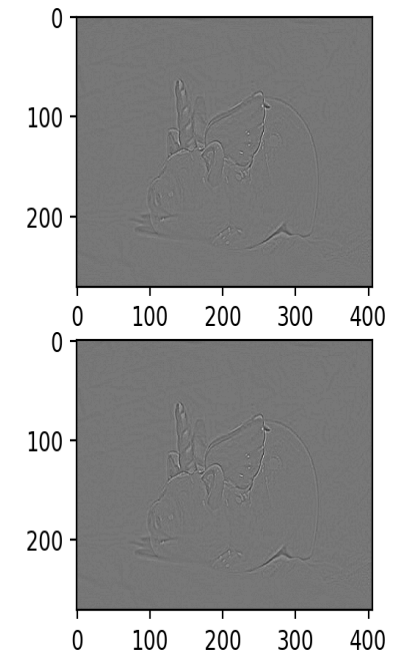
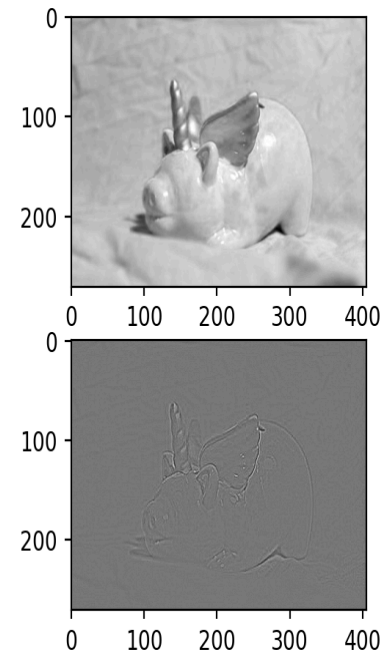
```
1 from skimage.filters import sobel_h, sobel_v,
2
3 edges=[]
4 edges.append(sobel(red))
5 edges.append(sobel_h(red))
6 edges.append(sobel_v(red))
7
8 fig,axs=plt.subplots(2,2,figsize=(10,4))
9 axs[0,0].imshow(red,cmap='gray')
10 j=0
11 for ax in axs.flatten()[1:]:
12     ax.imshow(edges[j],cmap='gray')
13     j+=1
```



Laplacian

- Laplacian filter approximates second derivatives

```
1 from skimage.filters import laplace
2
3 diff=[]
4 for sig in [3,5,9]:
5     diff.append(laplace(red, ksize=sig))
6
7 fig,axs=plt.subplots(2,2,figsize=(10,4))
8 axs[0,0].imshow(red,cmap='gray')
9 j=0
10 for ax in axs.flatten()[1:]:
11     ax.imshow(diff[j],cmap='gray')
12     j+=1
```

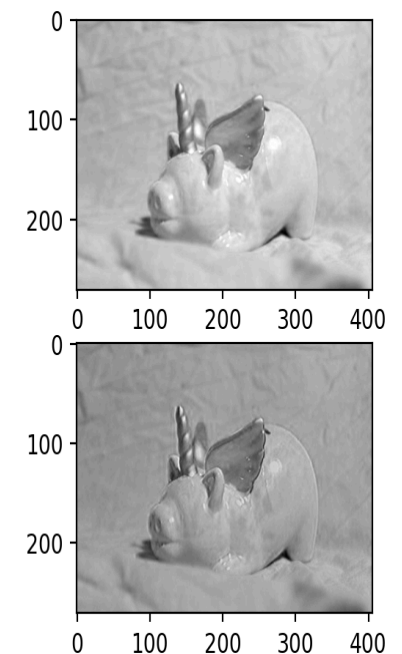
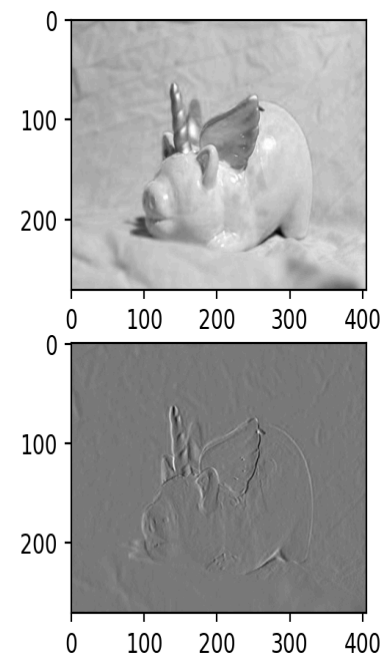


Sharpening via Convolution

Sharpening an image means creating a new image with the edges highlighted. In other words, this is somewhat similar to summing back the edges to the original image which we can do following the convolutional terminology.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

```
1 kernel = []
2 for i in range(3):
3     kernel.append(np.zeros((3,3)))
4 kernel[0][1,1]=1
5 kernel[1][1,2]=1
6 kernel[1][1,0]=-1
7 kernel[2]=kernel[0]+kernel[1]
8
9 fig,axs=plt.subplots(2,2,figsize=(10,4))
10 axs[0,0].imshow(red,cmap='gray')
11 j=0
12 for ax in axs.flatten()[1:]:
13     ax.imshow(convolve(red,kernel[j]),cmap='gray')
14     j+=1
```

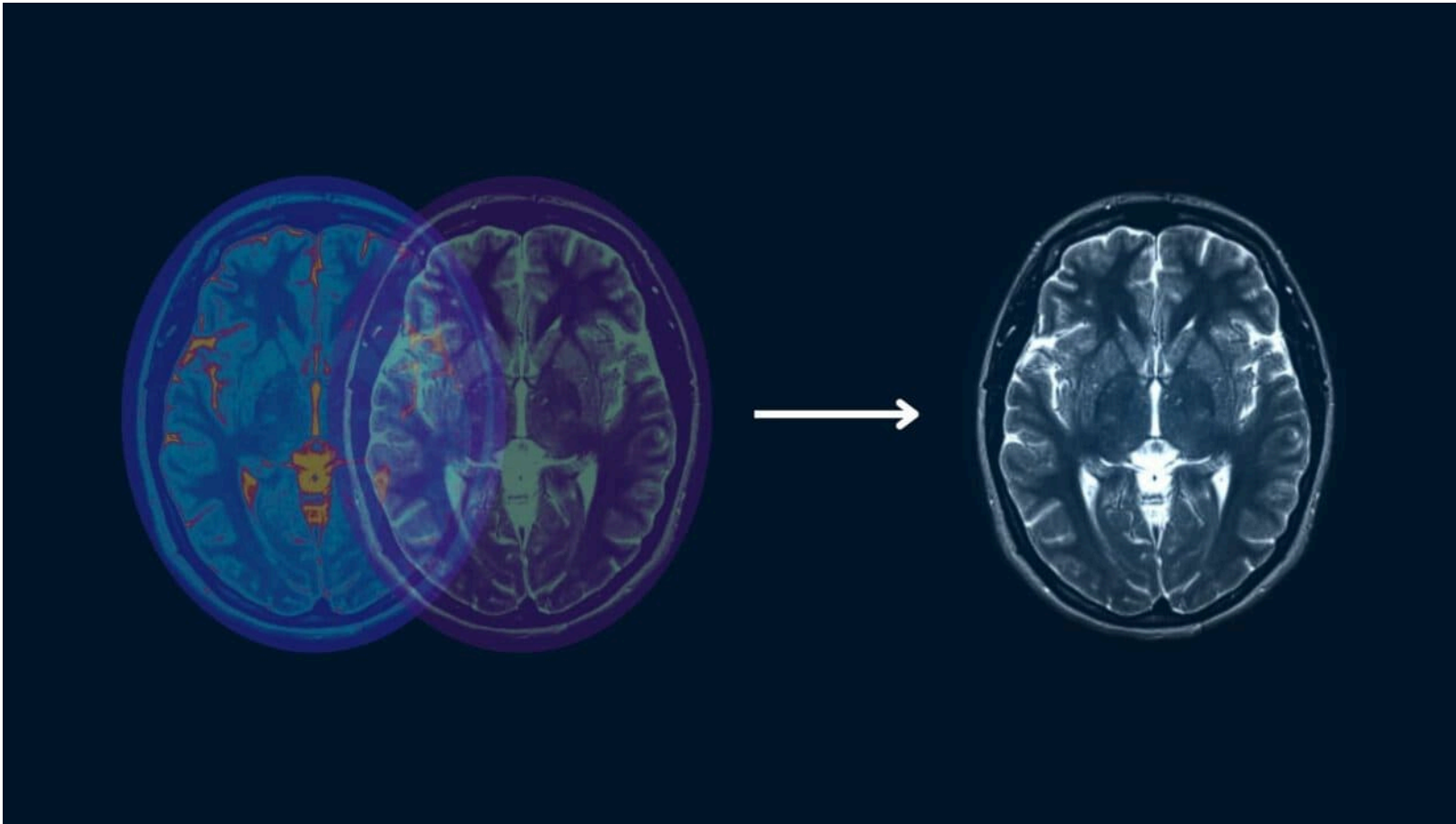


Unified View

- **Blurring, sharpening, edge detection, embossing...**
All are **convolution/correlation filters** with different kernels (or combinations of them).
- By choosing the right kernel:
 - Gaussian kernel → smoothing
 - Differential kernel → edges
 - Identity + negative weights → sharpening
 - Asymmetric kernel → directional effects (emboss, motion blur)

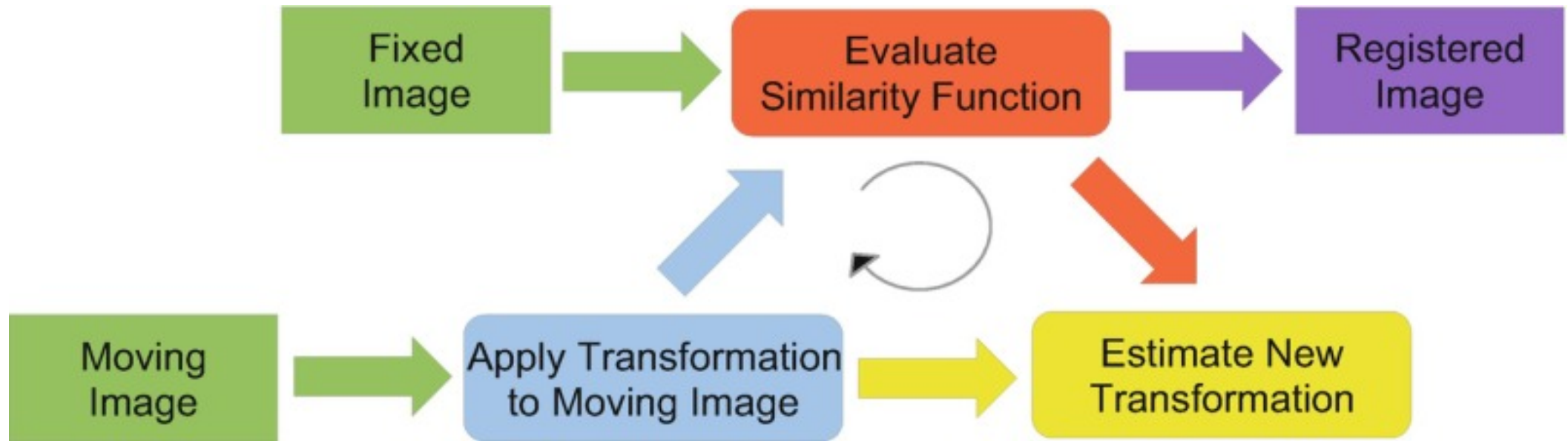
Convolution is the **unifying principle** behind most filters.

Registration



In many computer vision applications (e.g. object tracking and medical imaging) there is a need to align two or more images of the same object (or scene) taken from different perspectives, at different times, or in different conditions. Image registration algorithms transform a given image (a reference image) into another image (target image) so that they are geometrically aligned.

Registration process

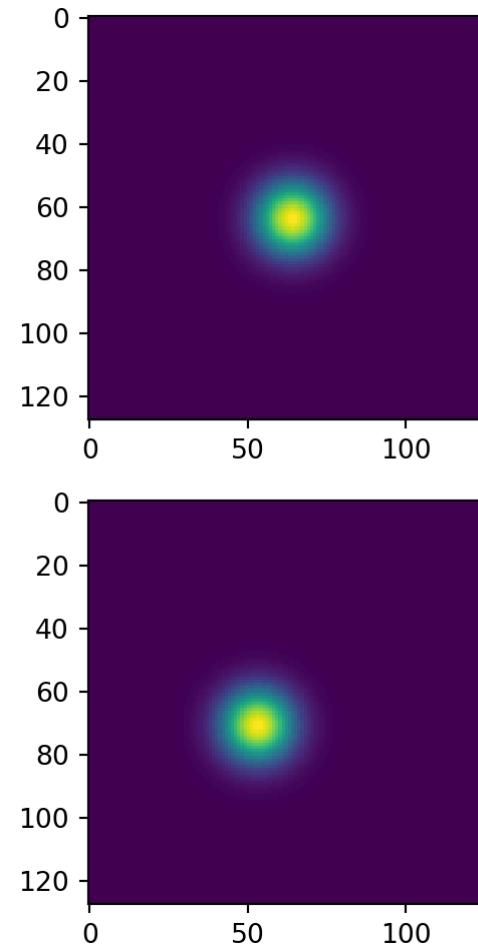


The main goal of an image registration algorithm is to take a moving image and transform it to be spatially or temporally aligned with a target fixed image. The algorithm is generally defined by two parts: the type of transformation allowed to be performed on the moving image (the transformation model) and a definition of good alignment (the similarity cost function) between the two images.

Registration at work

Let's suppose we have two images acquired by an optical system that are slightly shifted. I want to re-center them:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 # --- 1) Make two images: a Gaussian and a sh
4 def gaussian_2d(n=128, sigma=8.0, center=None
5     y, x = np.indices((n, n))
6     cy, cx = (n//2, n//2) if center is None e
7     return np.exp(-(((y - cy)**2 + (x - cx)**
8 n = 128
9 im1 = gaussian_2d(n, sigma=8.0, center=(64, 6
10 # Ground-truth integer shift (dy, dx). Positi
11 true_shift = (7, -11)
12 im2 = np.roll(np.roll(im1, true_shift[0], axi
13 fig,axs = plt.subplots(2,1,figsize=(6,6))
14 axs[0].imshow(im1)
15 axs[1].imshow(im2)
```



Cross-correlation

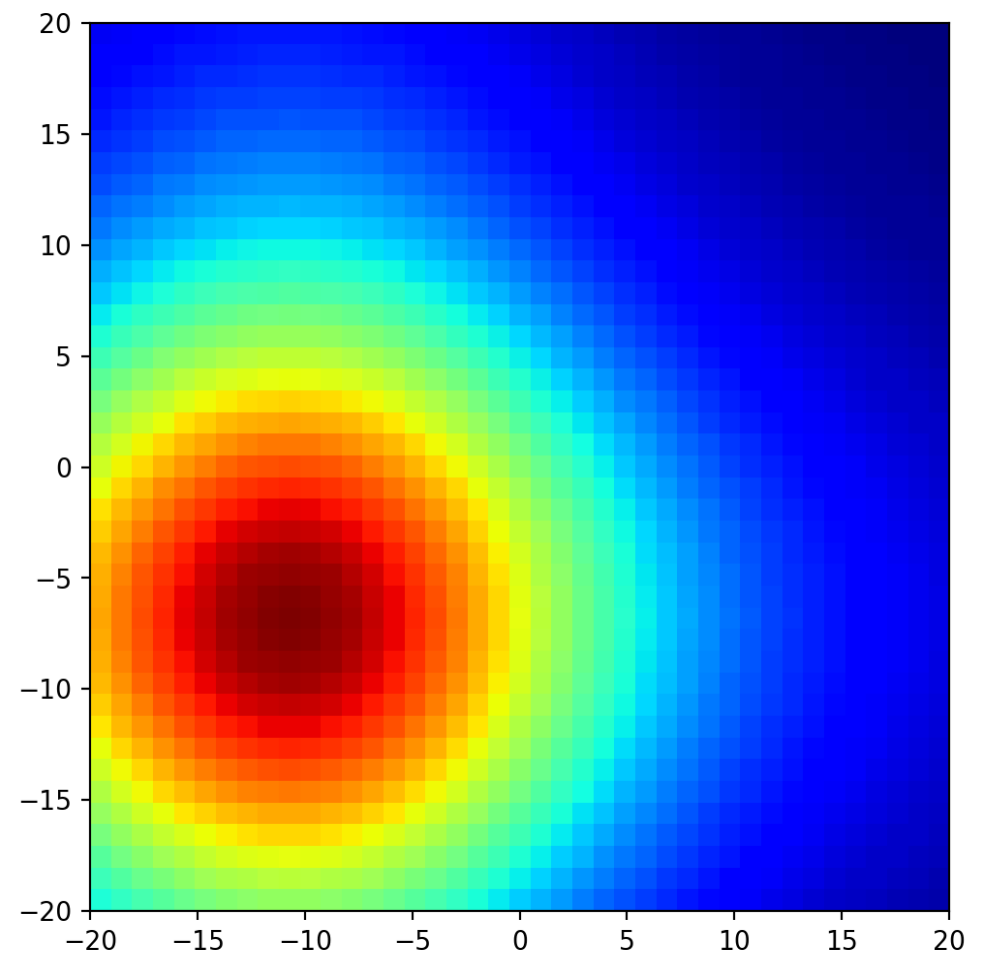
$$\text{NCC}(u_1, \dots, u_D) = \frac{\sum_{i_1, \dots, i_D} I_1(i_1, \dots, i_D) I_2(i_1 + u_1, \dots, i_D + u_D)}{\sqrt{\left\{ \sum_{j_1, \dots, j_D} [I_1(j_1, \dots, j_D)]^2 \right\} \left\{ \sum_{k_1, \dots, k_D} [I_2(k_1 + u_1, \dots, k_D + u_D)]^2 \right\}}},$$

```

1 def ncc(box1,box2):
2     box1 = box1
3     box2 = box2
4     num = np.sum(box1*box2)
5     den = np.sqrt(np.sum(box1**2)*np.sum(box2
6     return num/den
7
8 center = (64,64)
9 n=20
10 cc=np.zeros((2*n+1,2*n+1))
11 box1 = im1[center[0]-n:center[0]+n,center[1]-
12 for i in range(-n,n+1):
13     for j in range(-n,n+1):
14         box2 = im2[center[0]-n+i:center[0]+n+
15         cc[i+n,j+n] = ncc(box1,box2)
16
17 fig,ax = plt.subplots(1,1,figsize=(6,6))
18 x,y = np.unravel_index(np.argmax(cc, axis=None)
19 ax.imshow(cc,cmap='jet',extent=[-n,n,-n,n])
20 print(true_shift,(int(x)-n,int(y)-n))

```

(7, -11) (7, -11)



Registration in scikit-image

Scikit-Image provides image registration algorithms that can work perform the task on more complex images and when we are not just looking for a shift, but for an actual registration:

Registration:

- `optical_flow_ilk`
- `optical_flow_tvl1`
- `phase_cross_correlation`

Note

Image registration can be very useful also for pattern localisation. See Scipy [correlate2d](#) example.

Assignment

Program a jupyter notebook that does the following steps. Note that only numpy, matplotlib, scikit-image and scipy are allowed, **no other libraries!**

- Opens the provided image (cells.tif) and show the channels with an appropriate LUT to highlight specific features
- Using convolutions, suggest a kernel to sharpen the green channel and apply it to the image.
- Cut out (=make black) the saturated red region and show the image
- Recombine the channels swapping blue and green and show the obtained image

All choices should be **commented and justified** within the notebook (use markdown cells for this).

Deadline: 15/10/2025 at h 17:00

Late penalties: cap to 4* same day; cap to 3* afterwards